PROBLÈME · ANNALES SPÉ · 50 MIN

Décodage par l'algorithme de Viterbi

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Viterbi. Bob observe `Obs` $= [\text{obs}_0, \dots, \text{obs}_{N-1}]$, symboles de $\Sigma = [\![0, K-1]\!]$. $E[i][j]$: probabilité d'observer i sachant que j a été émis, ici $E = [[0.7, 0.2, 0.3], [0.2, 0.7, 0.1], [0.1, 0.1, 0.6]]$. $P[i][j]$: probabilité d'émettre j à l'instant $t{+}1$ sachant que i a été émis à l'instant t, ici $P = [[0.3, 0.2, 0.5], [0.4, 0.4, 0.2], [0.2$

Graphe : un sommet $S_{i,j}$ par symbole i et par instant j ; arcs de chaque $S_{i,j}$ vers tous les $S_{k,j+1}$; une source σ reliée à la première couche et une cible τ reliée à la dernière. Poids : $\sigma \to S_{i,0}$: $E_{\text{obs}_0,i}$; $S_{i,j} \to S_{k,j+1}$: $E_{\text{obs}_{j+1},k} P_{i,k}$; vers τ : 1. La probabilité d'un chemin est le produit des poids ; on cherche le chemin de probabilité maximale.

1. Nombre de sommets et d'arcs (sans σ, τ et leurs arcs).
2. Pour `Obs = [2,0]`, construire le graphe pondéré.
3. Nombre de chemins de σ à τ ; une exploration exhaustive est-elle envisageable ?
4. Écrire `maximumListe(liste) -> (float, int)` : maximum et plus petit indice où il est atteint.
5. Écrire `glouton(Obs, P, E, K, N) -> [int]`, qui suit à chaque étape l'arc de plus forte probabilité (on dispose de `initialiserGlouton`).
6. Complexité de l'approche gloutonne.
7. Chemin glouton sur la figure 4 (arcs $\sigma \to 0 : 0{,}6$, $\sigma \to 1 : 0{,}4$; $0 \to 0 : 0{,}5$, $0 \to 1 : 0{,}1$, $1 \to 0 : 0{,}9$, $1 \to 1 : 0{,}01$; arcs vers $\tau : 1$). Optimalité ?
8. Ramener la recherche à un plus court chemin à poids positifs ; quel algorithme ?

$T_{i,0} = E_{\text{obs}_0,i}$ et, pour $j \geqslant 1$, $T_{i,j} = \max_k T_{k,j-1} P_{k,i} E_{\text{obs}_j,i}$; `argT` garde l'indice k réalisant le maximum (-1 pour $j = 0$). On dispose de `initialiserViterbi(E, Obs0, K, N)` qui crée `T` et `argT` (K lignes, N colonnes) avec la colonne 0 remplie.

1. Écrire `construireTableauViterbi(Obs, P, E, K, N)` (de bas en haut).
2. Avec les tableaux donnés (dernière colonne de `T` : $1{,}8 \cdot 10^{-5}$, $3{,}5 \cdot 10^{-6}$, $1{,}3 \cdot 10^{-5}$; `argT` lignes : $[-1, 2, 0, 0, 2, 1, 1, 0]$, $[-1, 2, 2, 2, 2, 1, 1, 0]$, $[-1, 2, 0, 0, 2, 1, 1, 0]$), donner la séquence la plus probable.
3. Complexités temporelle et spatiale de la programmation dynamique.